

Chapter 3

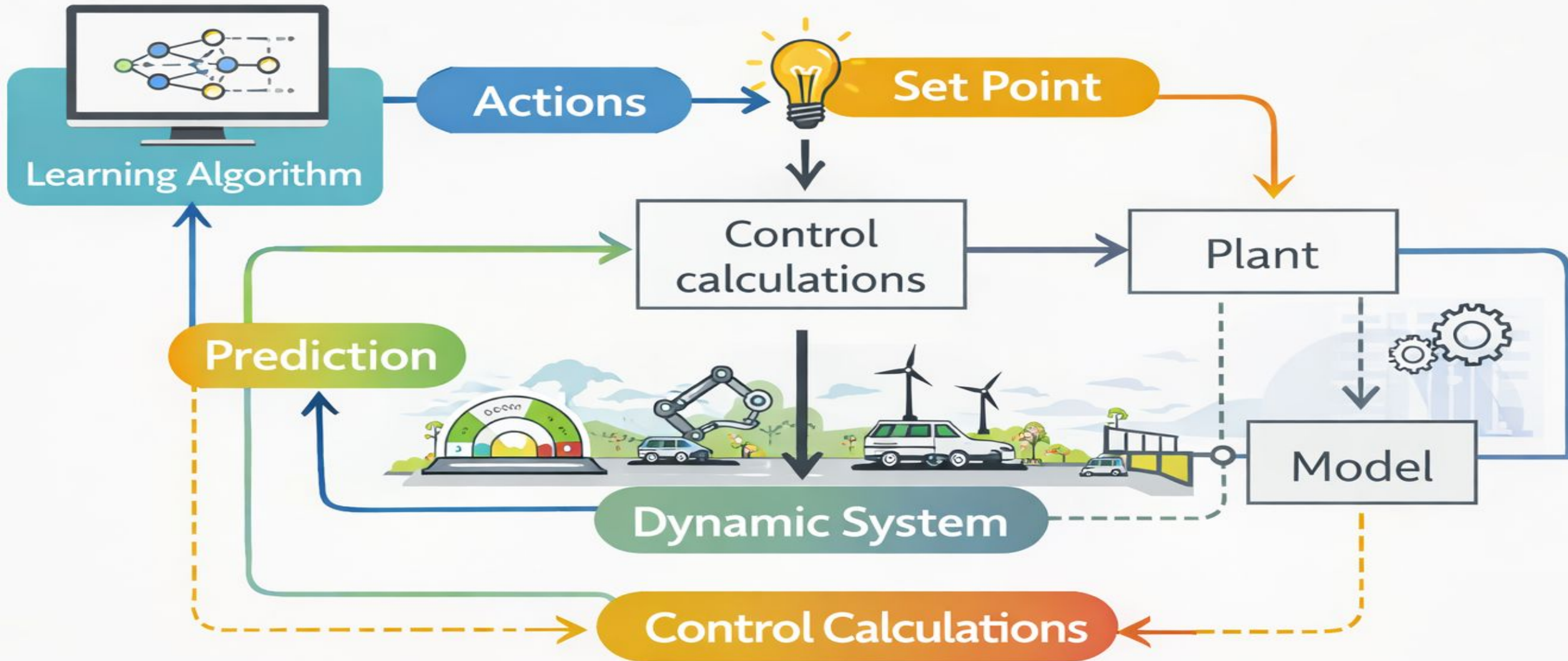
REINFORCEMENT ALGORITHMS AND APPLICATIONS

Algorithms for control learning

- ❑ Algorithms for Control Learning are methods that learn how to control a dynamic system by interacting with it and improving decisions over time.
- ❑ It is widely used in robotics, autonomous systems, traffic control, energy systems, and industrial automation.

Control Learning of Dynamic Systems

Learning algorithms control dynamic systems by interacting with them and improving decisions over time.



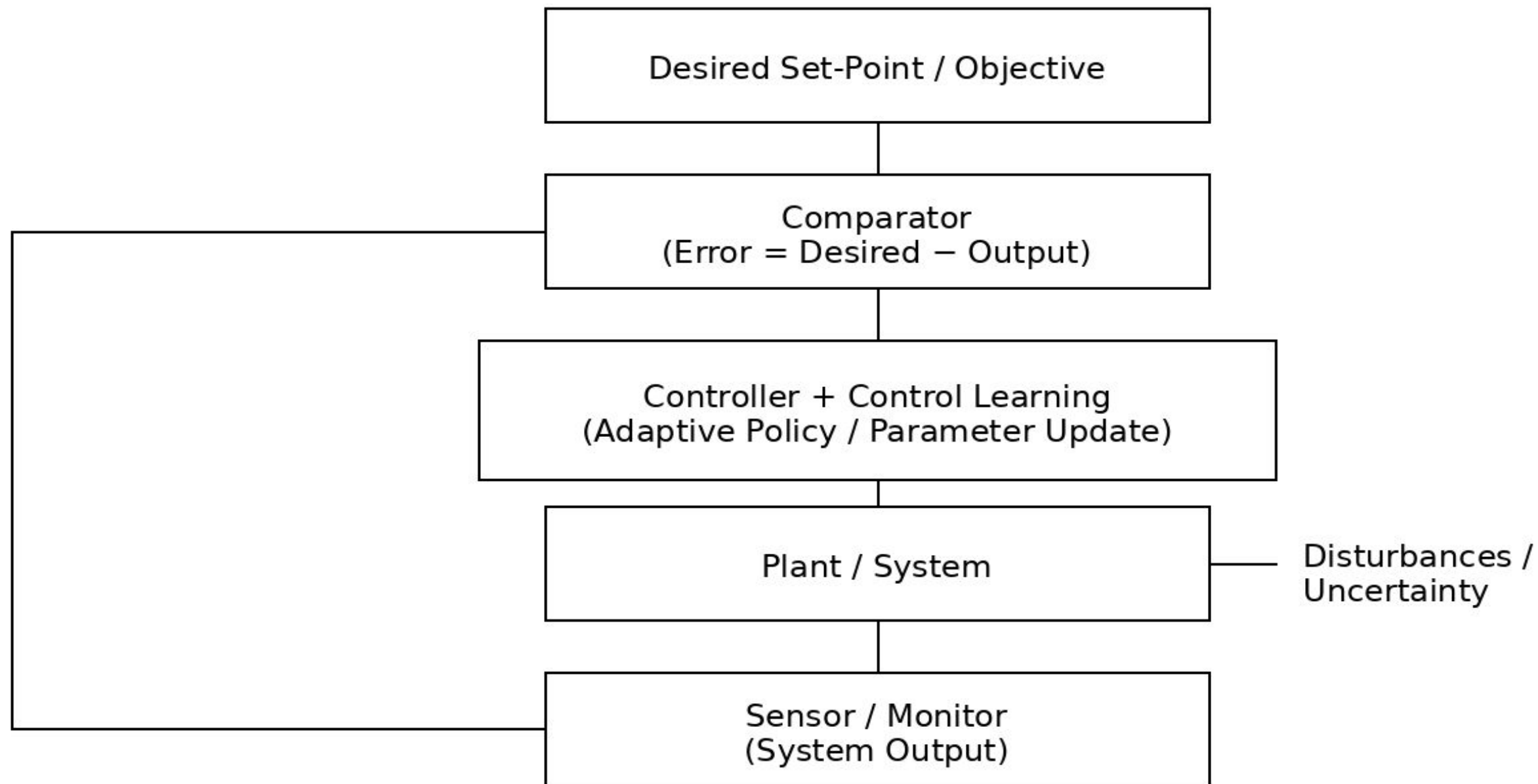
1) Learning through Continuous Interaction with Dynamic Systems

- Control learning algorithms function by continuously interacting with a dynamic system rather than relying on fixed control rules.
- The controller observes the system's state and applies control actions to influence its behavior.
- The system responds to these actions, generating feedback that reflects the quality of the control decision.
- This feedback is used to update the control strategy iteratively.
- Over time, the algorithm improves its ability to regulate the system under varying conditions.

2) Closed-Loop Feedback and Adaptation Mechanism

- A fundamental characteristic of control learning is the use of closed-loop feedback.
- System outputs are continuously monitored and compared with desired set-points or objectives.
- Any deviation or error is fed back into the controller for corrective action.
- This allows the algorithm to adapt to disturbances, uncertainties, and parameter variations.
- Closed-loop learning ensures robustness and long-term stability of the controlled system.

Closed-Loop Feedback Control Learning System



3) Optimization via Objective or Reward Functions

- Control learning algorithms are guided by an objective function or reward signal that quantifies performance.
- The objective may represent goals such as minimizing energy consumption, tracking error, or response time.
- Each control action is evaluated based on its contribution to the long-term objective.
- The learning algorithm adjusts its policy to maximize rewards or minimize costs over time.
- This optimization framework enables systematic improvement of control performance.

4) Exploration and Exploitation Trade-off in Control Decisions

- To learn an effective control policy, the algorithm must explore different control actions.
- Exploration helps the system discover new strategies that may outperform current solutions.
- At the same time, exploitation uses known successful actions to maintain stable operation.
- Excessive exploration can degrade performance, while excessive exploitation can lead to suboptimal control.
- A balanced trade-off ensures efficient learning without sacrificing system safety or stability.

❑ **Online and Offline Learning Capability**

In online learning, the controller updates its policy in real time while interacting with the system.

Offline learning uses historical or simulated data to train the control policy before deployment.

❑ **Model-Free and Model-Based Control Learning**

Model-free methods learn control policies directly from data without explicit system models.

Model-based approaches use or learn a system model to predict future behavior and plan optimal actions.

❑ Handling Uncertainty and Disturbances

Control learning algorithms are designed to operate under uncertain and noisy conditions.

They adapt control actions to compensate for disturbances, sensor noise, and unmodeled dynamics.

❑ **Stability and Convergence Considerations**

Maintaining system stability during the learning process is a major requirement. Convergence analysis ensures that the learned control policy reaches a stable and optimal solution.

❑ **Continuous State and Action Spaces**

Many real-world control systems involve continuous variables rather than discrete states.

Control learning algorithms use function approximation techniques such as neural networks to handle continuity.

❑ **Safety-Aware Control Learning**

Safety constraints are incorporated to prevent the system from entering unsafe states during learning.

Techniques like safe reinforcement learning and constraint-based control ensure reliable operation.

□ **Computational Efficiency and Real-Time Constraints**

Control decisions must be computed within strict time limits for real-time systems.

Efficient algorithms and hardware acceleration are used to meet timing and performance requirements.

□ **Adaptability and Generalization**

Control learning algorithms adapt to changing system dynamics and operating conditions.

They aim to generalize learned policies across different environments and scenarios.

Major Categories of Control Learning Algorithms

1) Reinforcement Learning Based Control

These methods frame control as a sequential decision problem and learn a policy that maximizes cumulative reward.

- ✓ Value-based: Q-learning, SARSA
- ✓ Policy-based: Policy Gradient, REINFORCE
- ✓ Actor–Critic: A2C, A3C, PPO

Applications: traffic signal control, robotics, energy management, autonomous vehicles.

2) Adaptive Control

The controller adjusts its parameters online to cope with unknown or changing system dynamics.

- Model Reference Adaptive Control (MRAC)
- Self-tuning regulators

Applications: industrial processes, aerospace systems.

3) Optimal Control Learning

Learns control laws that minimize a cost function over time.

- ☐ Dynamic Programming
- ☐ Approximate Dynamic Programming (ADP)
- ☐ Learning-based MPC

Applications: robotics motion planning, economic dispatch.

4) Neural Network–Based Control

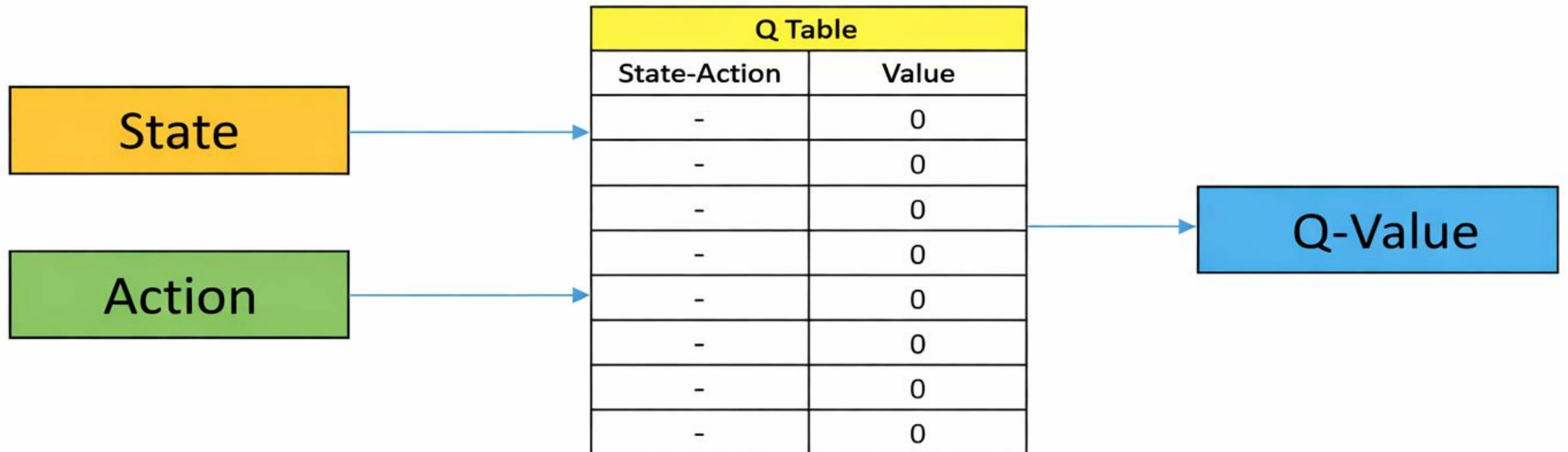
Uses neural networks to approximate system models or control policies.

- Neuro-control
- Adaptive neural controllers

Applications: nonlinear and complex systems.

Q-learning

- ❑ **Q-learning** is a model-free reinforcement learning algorithm that learns the optimal action for each state through trial-and-error interaction with the environment.
- ❑ It updates an action–value function $Q(s, a)$ using immediate rewards and the maximum expected future reward.
- ❑ Over time, Q-learning converges to an optimal policy that maximizes long-term cumulative reward without requiring a system model.



Core idea

- ❑ The agent keeps a table (or function) $Q(s, a)$ that estimates how good it is to take action a in state s .
- ❑ By repeatedly acting, observing rewards, and updating Q-values, the agent converges to an **optimal policy** that maximizes long-term reward.

Key components

- **State (s):** Current situation of the environment.
- **Action (a):** Choice the agent can make in a state.
- **Reward (r):** Immediate feedback after an action.
- **Policy (π):** Strategy that selects actions based on Q-values.
- **Learning rate (α):** How fast new information overrides old estimates.
- **Discount factor (γ):** How much future rewards matter.

Q-learning update rule

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- ❑ Start with the old estimate $Q(s, a)$.
- ❑ Look at the **reward now** plus the **best possible future value** from the next state s' .
- ❑ Nudge the old estimate toward this target by a fraction α .

Algorithm steps (concise)

1. Initialize $Q(s, a)$ arbitrarily (often zeros).
2. Observe the current state s .
3. Choose an action a (commonly ϵ -greedy for exploration).
4. Execute a , receive reward r and next state s' .
5. Update $Q(s, a)$ using the update rule.
6. Set $s \leftarrow s'$ and repeat until convergence.

Exploration vs. exploitation

- **Exploration:** Try new actions to discover better rewards (ϵ -greedy picks random actions with probability ϵ).
- **Exploitation:** Choose the action with the highest Q-value.
- A decaying ϵ balances learning early and performance later.

Applications

- Traffic signal control
- Robot navigation (grid worlds)
- Elevator scheduling
- Game playing (classic control tasks)

Why Q-learning is popular

- **Model-free:** No system model needed.
- **Off-policy:** Learns the optimal policy even while exploring with a different behavior policy.
- **Simple & robust:** Works well for many discrete control problems.

Limitations

- **State explosion:** Q-tables don't scale to large or continuous spaces.
- **Slow convergence:** Can require many interactions.
- **Safety:** Pure exploration may be risky in real systems.

Common fix: Use function approximation → **Deep Q-Networks (DQN)**.

Traffic Signal Control

- ❑ A traffic-signal control agent is operating at an intersection and is currently observing **medium traffic conditions**, represented by the state s_2 . Based on its current policy, the agent selects the action a_1 , which corresponds to **keeping the signal green** for the current direction.
- ❑ After executing this action, the traffic flow improves, and the environment transitions to a **normal traffic state**, denoted as s_1 . As a result of this favorable transition, the agent receives a **positive reward**, reflecting improved traffic movement and reduced waiting time.
- ❑ This experience is then used by the agent to update its action-value estimate, reinforcing the effectiveness of keeping the signal green in medium traffic conditions.

Traffic Signal Control

A traffic-signal agent is currently in Medium traffic state s_2 .

It chooses action a_1 = Keep Green.

After acting, traffic improves to Normal state s_1 , and the agent receives a positive reward.

Problems statement :

- Given a reinforcement learning scenario based on the Q-learning framework, the learning parameters are defined as follows: the learning rate is set to $\alpha = 0.5$, and the discount factor is $\gamma = 0.8$.
- The current Q-table indicates that, for the present state–action pair (s_2, a_1) , the Q-value is $Q(s_2, a_1) = 5$. After executing action a_1 in state s_2 , the agent transitions to the next state s_1 , where two possible actions are available.
- The corresponding Q-values in the next state are given as $Q(s_1, a_1) = 6$ and $Q(s_1, a_2) = 8$.
- Using these values, the objective is to update the Q-value of the current state–action pair according to the Q-learning update rule by incorporating the learning rate, discount factor, and the maximum expected future reward from the next state.

Given Values

Learning parameters

- Learning rate: $\alpha = 0.5$
- Discount factor: $\gamma = 0.8$

Current Q-values

Assume the current Q-table has:

- Current state-action value: $Q(s_2, a_1) = 5$
- Next state s_1 has two possible actions:

$$Q(s_1, a_1) = 6, Q(s_1, a_2) = 8$$

So: $\max_{a'} Q(s_1, a') = \max(6, 8) = 8$

Observed reward

Agent receives: $r = 4$

Step-by-Step Derivation

Step 1: Compute the TD target Target = $r + \gamma \max_{a'} Q(s', a')$

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Step 1: Compute

$$\text{Target} = r + \gamma \max_{a'} Q(s', a')$$

Substitute:

$$\text{Target} = 4 + 0.8 \times 8$$

$$\text{Target} = 4 + 6.4 = 10.4$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Step 2: Compute TD error

$$\delta = \text{Target} - Q(s, a)$$

$$\delta = 10.4 - 5 = 5.4$$

Step 3: Update Q-value

$$Q_{\text{new}}(s_2, a_1) = Q(s_2, a_1) + \alpha\delta$$

$$Q_{\text{new}}(s_2, a_1) = 5 + 0.5 \times 5.4$$

$$Q_{\text{new}}(s_2, a_1) = 5 + 2.7 = 7.7$$

Final Answer

$Q(s_2, a_1)$ updated from 5 to 7.7

Interpretation

- The future looks good because in the next state s_1 there is an action with high value (8).
- So, the current action a_1 becomes more valuable.
- Q-learning increased $Q(s_2, a_1)$ toward the target.

Discrete Action Space

Discrete action space

- A discrete action space in reinforcement learning means that the agent can choose actions from a finite, countable set of predefined options at each decision step.

If the action space is discrete, it can be written as:

$$A = \{a_1, a_2, a_3, \dots, a_n\}$$

Characteristics

- ✓ Finite number of actions
- ✓ Easy to enumerate and compare
- ✓ Suitable for **Q-learning, SARSA, and tabular methods**
- ✓ Often used in **games and simple control systems**

SARSA (State–Action–Reward–State–Action)

SARSA is an on-policy reinforcement learning algorithm used to learn the action-value function $Q(s, a)$. The name comes from the sequence of variables used in its update:

$$\boxed{S \rightarrow A \rightarrow R \rightarrow S \rightarrow A}$$

SARSA updates the Q-value using the actual action chosen in the next state, not the maximum possible action. That is why it is called an on-policy method.

SARSA Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Where:

- α = learning rate
- γ = discount factor
- R_{t+1} = reward
- $Q(S_{t+1}, A_{t+1})$ = value of the actual next action

Example (Traffic Signal)

Current state:

$$S_t = s_2 \text{ (Medium traffic)}$$

Action chosen:

$$A_t = a_1 \text{ (Keep Green)}$$

Reward received:

$$R_{t+1} = 4$$

Next state:

Next action (chosen by policy):

$$S_{t+1} = s_1$$

$$A_{t+1} = a_2$$

Assume:

- $Q(s_2, a_1) = 5$
- $Q(s_1, a_2) = 8$
- $\alpha = 0.5$
- $\gamma = 0.8$

$$\text{Update: } Q_{\text{new}} = 5 + 0.5[4 + 0.8(8) - 5]$$

$$= 5 + 0.5(4 + 6.4 - 5)$$

$$= 5 + 0.5(5.4)$$

$$= 5 + 2.7$$

$$\boxed{Q(s_2, a_1) = 7.7}$$

Difference Between SARSA and Q-Learning

Feature	SARSA	Q-Learning
Type	On-policy	Off-policy
Uses	$Q(S', A')$	$\max Q(S', a')$
Behavior	Safer, conservative	More aggressive
Risk handling	Learns actual behavior	Learns optimal behavior

Temporal Difference (TD) Learning

Temporal Difference (TD) learning is a reinforcement learning method that updates value estimates by using the difference between successive predictions.

It combines ideas from:

- Monte Carlo methods (learning from actual rewards)
- Dynamic Programming (bootstrapping using current value estimates)

Instead of waiting until the end of an episode, TD learning updates the value estimate immediately after each step, using the observed reward and the estimated value of the next state.

Meaning of TD Error $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$

$\delta_t = \text{Target} - \text{Current Estimate}$

Example (Traffic)

State: s_2 (Medium traffic)

- Assume:
- $V(s_2) = 5$
 - Reward received $R = 4$
 - Next state s_1
 - $V(s_1) = 8$
 - $\gamma = 0.8$
 - $\alpha = 0.5$

Example (Traffic)

State: s_2 (Medium traffic)

- Assume:
- $V(s_2) = 5$
 - Reward received $R = 4$
 - Next state s_1
 - $V(s_1) = 8$
 - $\gamma = 0.8$
 - $\alpha = 0.5$

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

Step 1: Compute TD Target $4 + 0.8(8) = 4 + 6.4 = 10.4$

Step 2: Compute TD Error $\delta = 10.4 - 5 = 5.4$

Step 3: Update Value $V_{\text{new}}(s_2) = 5 + 0.5(5.4)$
 $= 5 + 2.7 = 7.7$

Lambda (λ)

Lambda (λ) in Reinforcement Learning

In reinforcement learning, lambda (λ) is a parameter used in Temporal Difference (TD) learning to balance between:

- Monte Carlo learning
- TD(0)

It appears in methods such as:

- **TD(λ)**
- **SARSA(λ)**
- **Q(λ)**

❑ **Lambda controls how much past states/actions are credited when a reward is received.**

$$0 \leq \lambda \leq 1$$

Intuitive Meaning

- $\lambda = 0$
→ Pure TD(0) (only immediate next step considered)
- $\lambda = 1$
→ Monte Carlo method (full return used)
- $0 < \lambda < 1$
→ A mix of short-term and long-term credit assignment

Why Is Lambda Needed?

- Rewards are delayed
- We need to assign credit to earlier decisions
- One-step updates are too slow
- Full-episode updates are too noisy
- Lambda provides a smooth balance.

Difference

Method	Lambda Value
<u>TD(0)</u>	0
Monte Carlo	1
TD(λ)	Between 0 and 1

Deep Q-Network (DQN)

Deep Q-Network (DQN) is a reinforcement learning algorithm that uses a deep neural network to approximate the Q-value function in large or continuous state spaces.

Deep Q-Network (DQN)

- ❑ A Deep Q-Network (DQN) is a reinforcement learning algorithm that combines:
 - ❑ Q-learning (value-based RL method)
 - ❑ Deep Neural Networks (function approximation)
- ❑ It is used when the state space is large or continuous, where storing a Q-table is impossible.

DEEP Q-NETWORK (DQN)

Reinforcement Learning Algorithm

Q-Learning

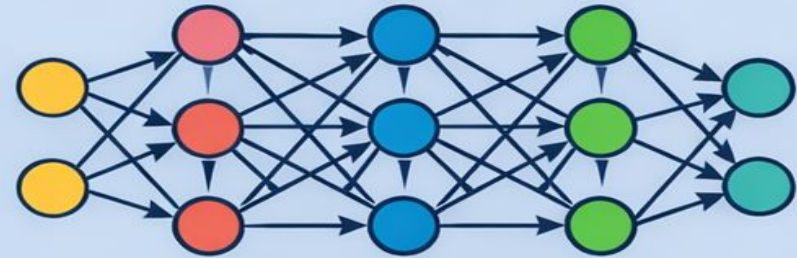
Value-Based RL Method

State	Q-Values
State 1	[2.5, 3.1]
State 2	[1.2, 4.0]
State 3	[0.8, 2.7]

Function Approximation

Deep Neural Network

Function Approximation



Large or Continuous State Space

Complex Environment



Game Screen



Robotics Control



Too large for Q-Table! Use Deep Neural Networks!

Why Do We Need DQN?

- ❑ In classical Q-learning:

$$Q(s, a)$$

- ❑ It is stored in a table.

- ❑ But if:

- ❑ States are images (like video frames),
- ❑ States are continuous values,
- ❑ State space is very large,

- ❑ Then:

- ❑ Q-table becomes impossible to store.

- ❑ So we use a neural network to approximate $Q(s, a)$.

Concept Idea

- ❑ Instead of storing values in a table:

$$Q(s, a) \approx Q(s, a; \theta)$$

- ❑ where:

- θ = neural network parameters

- ❑ The neural network takes the state as input and outputs Q-values for all actions.

DQN Update Rule

$$L(\theta) = \left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2$$

Where:

- θ = current network parameters
- θ^- = target network parameters
- r = reward
- γ = discount factor

Two Key Innovations in DQN

1) Experience Replay

- Stores past experiences in memory
- Randomly samples mini-batches
- Breaks correlation between samples
- Stabilizes training

2) Target Network

- Uses a separate network for target calculation
- Updated periodically
- Prevents oscillations

Difference: Q-Learning vs DQN

Q-Learning	DQN
Uses Q-table	Uses neural network
Small state space	Large state space
Tabular	Function approximation
Simple problems	Complex problems (robotics)

- **Problem statement**: At a particular learning step in a Deep Q-Network (DQN), assume the traffic control agent is currently observing a state described as “medium congestion.” In this state, the agent selects the action a_2 , which corresponds to switching the signal, and after performing this action, it receives a reward of 7. The discount factor is given as $\gamma = 0.8$, meaning the agent values future rewards but slightly less than immediate rewards.
- Next, the agent looks at the next state s' and uses the target network to estimate the Q-values for all possible actions in that next state. The target network predicts that $Q(s', a_1; \theta^-) = 9$ and $Q(s', a_2; \theta^-) = 11$. Since DQN follows the Q-learning principle, it selects the maximum of these values to estimate the best possible future reward. Therefore, $\max_{a'} Q(s', a'; \theta^-) = 11$.
- This value (11) represents the best expected future return from the next state. The agent then combines the immediate reward (7) with the discounted future reward (0.8×11) to compute the target for updating the network. In simple terms, the agent updates its neural network so that the predicted Q-value for taking action a_2 in the “medium congestion” state moves closer to this improved target estimate. This process allows the Deep Q-Network to gradually learn which actions produce the highest long-term traffic efficiency.

example: DQN learning step

Assume at a particular moment:

- Current state: s = "medium congestion"
- Action taken: $a = a_2$ (Switch)
- Observed reward: $r = 7$
- Discount: $\gamma = 0.8$

Target-network outputs at next state s'

$$Q(s', a_1; \theta^-) = 9, \quad Q(s', a_2; \theta^-) = 11$$

$$\text{So:} \quad \max_{a'} Q(s', a'; \theta^-) = 11$$

Step A: Compute the target

$$\begin{aligned}y &= r + \gamma \max_{a'} Q(s', a'; \theta^-) \\&= 7 + 0.8 \times 11 = 7 + 8.8 \\&= 15.8\end{aligned}$$

Step B: Current network prediction

Suppose the current network predicts: $Q(s, a_2; \theta) = 12.0$

Step C: TD error $\delta = y - Q(s, a_2; \theta)$

$$\begin{aligned}&= 15.8 - 12.0 \\&= 3.8\end{aligned}$$

Step D: Loss $L(\theta) = \delta^2$

$$\begin{aligned}&= (3.8)^2 \\&= 14.44\end{aligned}$$

Step E: Parameter update

Since $\delta > 0$, the update will **increase** $Q(s, a_2; \theta)$ for similar states s .

Meaning: **"Switch"** becomes more likely to be chosen in such traffic conditions.

Continuous action space:

❑ Continuous Action Space

- ❑ A continuous action space means the agent's action is not chosen from a small fixed list, but can take any real value within a range.

(discrete) $A = \{a_1, a_2, \dots, a_n\}$

real numbers.

(continuous), e.g. $A \subseteq \mathbb{R}^k$

$$\mathbb{R} = \{\dots, -2, -1.5, 0, 3.14, 100, \dots\}$$

$$a \in [-1, 1] \quad \text{or} \quad a = (a_1, a_2) \in \mathbb{R}^2$$

k-dimensional real space.

Steering angle is the angle by which the front wheels turn.

It controls the **direction** of the vehicle.

$$\theta \in [-30^\circ, +30^\circ]$$

- $0^\circ \rightarrow$ go straight
- Positive angle $\rightarrow$ turn right
- Negative angle $\rightarrow$ turn left

- ❑ **Why Q-learning struggles in continuous actions**

- ❑ **Q-learning needs:**

$$\max_{a'} Q(s', a')$$

- ❑ But if a' is continuous (infinite possibilities), taking a max over infinite actions is hard.

- ❑ So for continuous actions we typically use policy-based / actor–critic methods.

Algorithms commonly used

- Policy Gradient
- Actor–Critic
- DDPG (continuous version of Q-learning idea)
- TD3
- SAC
- PPO (also works in continuous)

Deep Deterministic Policy Gradient

What is DDPG?

- ❑ Deep Deterministic Policy Gradient (DDPG) is a model-free, off-policy, actor–critic reinforcement learning algorithm designed for continuous action spaces.
- ❑ It combines:
 - ❑ Deep Q-Network (DQN) ideas (experience + target networks)
 - ❑ Deterministic Policy Gradient (DPG) for continuous control

DDPG is widely used in:

- Robotics control
- Autonomous driving
- Energy systems
- Industrial process control
- Continuous signal optimization

Why DDPG is Needed?

- ❑ Traditional Q-learning works well for discrete actions.
- ❑ But in many real systems:
 - ❑ Steering angle $\in [-30^\circ, +30^\circ]$
 - ❑ Traffic green time $\in [10 \text{ sec}, 90 \text{ sec}]$
- ❑ These are continuous values.
- ❑ If we discretize them:
 - ❑ Action space becomes huge
 - ❑ Learning becomes unstable
- ❑ DDPG directly outputs continuous actions.

Components of DDPG

DDPG has four neural networks:

Network	Purpose
Actor	Outputs action
Critic	Estimates Q-value
Target Actor	Stabilizes learning
Target Critic	Stabilizes learning

DDPG Algorithm Steps

1. Initialize actor and critic networks
2. Initialize target networks
3. Initialize replay buffer
4. For each episode:
 - Observe state
 - Select action + exploration noise
 - Execute action
 - Store transition in buffer
 - Sample mini-batch
 - Update critic
 - Update actor
 - Soft update target networks

Complete Learning Flow

- Car observes deviation
- Actor outputs steering
- Car moves
- Reward computed
- Critic updates Q
- Actor updates policy
- Repeat until centered driving achieved

Problem

- Keep the car exactly at the **center of the lane** using continuous steering control. This is a **continuous control problem**, hence we use **DDPG** instead of Q-learning.

We model lane keeping as a Markov Decision Process (MDP).

State Space S

Let state be:

$$s_t = [d_t, v_t, \phi_t]$$

Where:

- d_t = lateral deviation from lane center (meters)
- v_t = velocity (m/s)
- ϕ_t = heading angle error (degrees)

Value

$$s_t = [0.3, 20, 5]$$

- 0.3 m right of center
- Speed = 20 m/s
- Heading angle error = 5°

Action Space $A \subset \mathbb{R}$

Action is steering angle:

$$a_t = \delta_t$$
$$\delta_t \in [-30^\circ, +30^\circ]$$

This is continuous, so Q-table is impossible.

Reward Function

We design reward to penalize deviation:

$$r_t = -(d_t^2 + 0.1\phi_t^2)$$

Why squared?

- Encourages small deviation strongly
- Penalizes large errors heavily
- Smooth gradient for learning

reward calculate

$$r_t = -(d_t^2 + 0.1\phi_t^2)$$

Value Assumed

If $d_t = 0.3,$

$$\phi_t = 5^\circ$$

$$r_t = -(0.3^2 + 0.1(5^2))$$

$$r_t = -(0.09 + 2.5)$$

$$r_t = -2.59$$

d_t = lateral deviation from lane center (meters)

v_t = velocity (m/s)

ϕ_t = heading angle error (degrees)

❑ The actor represents:

$$a_t = \mu(s_t \mid \theta^\mu)$$

Suppose actor outputs:

$$\delta_t = -6^\circ$$

❑ Steer slightly left to correct right deviation.

Vehicle Dynamics (Environment Transition)

$$\phi_{t+1} = \phi_t + \frac{v_t}{L} \tan(\delta_t) \Delta t$$

Where:

- L = wheelbase
- Δt = time step

Assume:

- $v_t = 20$
- $L = 2.5$
- $\Delta t = 0.1$
- $\delta_t = -6^\circ$

$$\phi_{t+1} = \phi_t + \frac{v_t}{L} \tan(\delta_t) \Delta t$$

$$\tan(-6^\circ) \approx -0.105$$

$$\phi_{t+1} = 5 + \frac{20}{2.5} (-0.105)(0.1)$$

$$= 5 + (8)(-0.105)(0.1)$$

$$= 5 - 0.084$$

$$\phi_{t+1} = 4.916^\circ$$

Correct direction.

Critic Network Update : Critic estimates:

$$Q(s_t, a_t)$$

Target value: $y_t = r_t + \gamma Q'(s_{t+1}, \mu'(s_{t+1}))$

Assume:

- $r_t = -2.59$
- $\gamma = 0.99$
- $Q'(s_{t+1}, \mu'(s_{t+1})) = 8.5$

$$y_t = -2.59 + 0.99(8.5)$$

$$y_t = -2.59 + 8.415$$

$$y_t = 5.825$$

Critic minimizes:

$$L = (Q(s_t, a_t) - 5.825)^2$$

Interpretation:

- ☐ **Critic tells actor:**
 "If you increase steering slightly left, Q increases"
- ☐ **Actor shifts policy toward that direction**

Learning Intuition

If deviation is positive (car right side):

Actor learns:

$$\delta < 0$$

Steer left.

If deviation is negative (car left side):

Actor learns:

$$\delta > 0$$

Steer right.

Asynchronous Action Space – Introduction with Example

In **Reinforcement Learning (RL)**, the **action space** defines all possible actions an agent can take.

Normally, in classical RL:

- At each time step **t** , the agent selects **one action**
- The environment updates immediately
- The next state is observed

But in many real-world systems (robotics, networking, distributed systems), actions may:

- Occur at **different time intervals**
- Be executed **independently**
- Not update simultaneously

Asynchronous Action Space

Asynchronous Action Space is a reinforcement learning setup where different actions operate independently at different time intervals instead of being executed simultaneously at each time step.

Asynchronous Action Space is a setting where:

- Multiple actions are **triggered independently**
- Actions may operate on **different clocks**
- The environment does **not wait** for all actions before updating
- State transitions may occur at different time scales

Formally: $a_t \in \mathcal{A}$

We have: $a_t^i \in \mathcal{A}_i$

Where:

- $i \rightarrow$ **Different agents or subsystems**
- Each action space $\mathcal{A}_i$ operates independently, Updates occur asynchronously

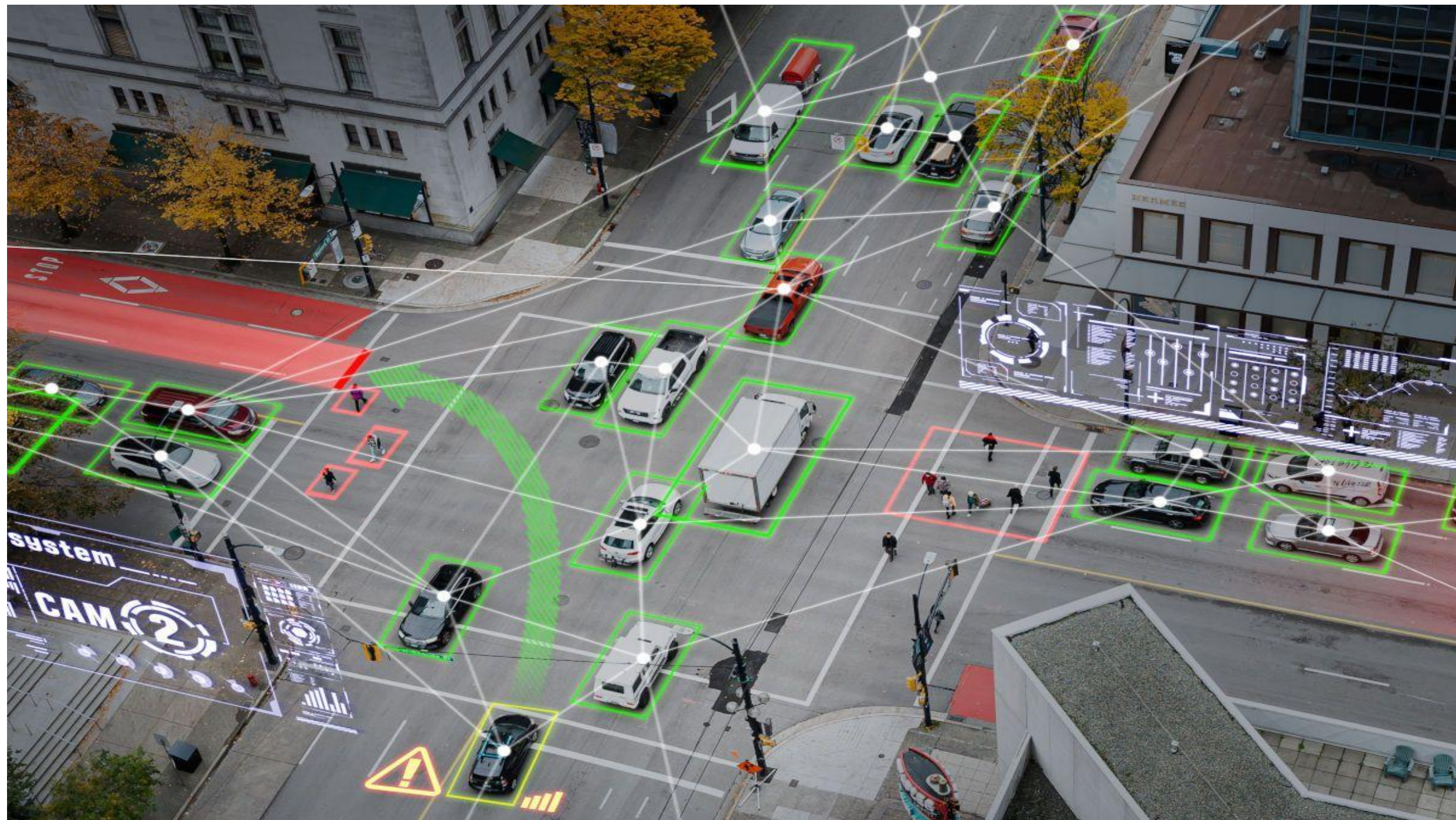
Real-Time Example 1: Smart Traffic Control

Scenario:

At an intersection:

- Traffic light controller updates every **30 seconds**
- Pedestrian crossing updates every **10 seconds**
- Emergency override system activates anytime

These actions do NOT synchronize.



Mathematical Representation

$s_t = (\text{traffic density, pedestrian count, emergency flag})$

Actions: $a_t^{(1)} = \text{Light timing adjustment}$

$a_t^{(2)} = \text{Pedestrian signal control}$

$a_t^{(3)} = \text{Emergency override}$

Each action updates at different time intervals:

$$t_1 \neq t_2 \neq t_3$$

State transition: $s_{t+1} = f(s_t, a_{t_1}^{(1)}, a_{t_2}^{(2)}, a_{t_3}^{(3)})$

Difference: Synchronous vs Asynchronous

Synchronous	Asynchronous
One action per time step	Multiple independent actions
Single clock	Multiple clocks
State updates uniformly	State updates irregularly
Easy modeling	More realistic

Where It Is Used

- Multi-agent systems
- Robotics
- Network traffic control
- Distributed computing
- Smart grids
- Edge computing

The **Asynchronous Advantage Actor–Critic (A3C)** algorithm is a reinforcement learning method that combines:

- **Actor–Critic architecture**
- **Advantage function**
- **Asynchronous parallel training**

It was introduced by **Volodymyr Mnih** and the **DeepMind** team in 2016.

A3C became famous because it:

- ☐ Does **not require replay memory**
- ☐ Uses **multiple parallel agents**
- ☐ Trains faster and more stably

Step-by-Step Working of A3C

1. Initialize global Actor–Critic network
2. Create multiple worker agents
3. Each worker:
 - Interacts with environment
 - Collects rewards
 - Computes gradients
 - Updates global network
4. Repeat until convergence

Asynchronous Advantage Actor–Critic (A3C)

Real-Time Example: Automatic (Self-Driving) Car Control

Example : Lane-Keeping & Speed Control System

Imagine an automatic car driving on highways.

It must:

- ❑ Maintain lane position
- ❑ Control steering
- ❑ Adjust speed
- ❑ Avoid collisions

Instead of one learning agent, **A3C uses multiple parallel cars (simulated environments)** to learn faster.

CARLA Environment



State s_t

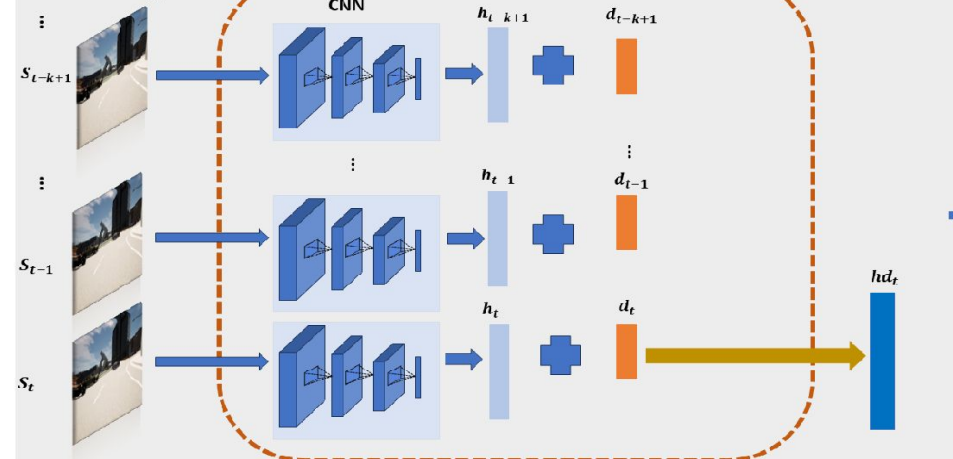
Reward r_t

Action a_t

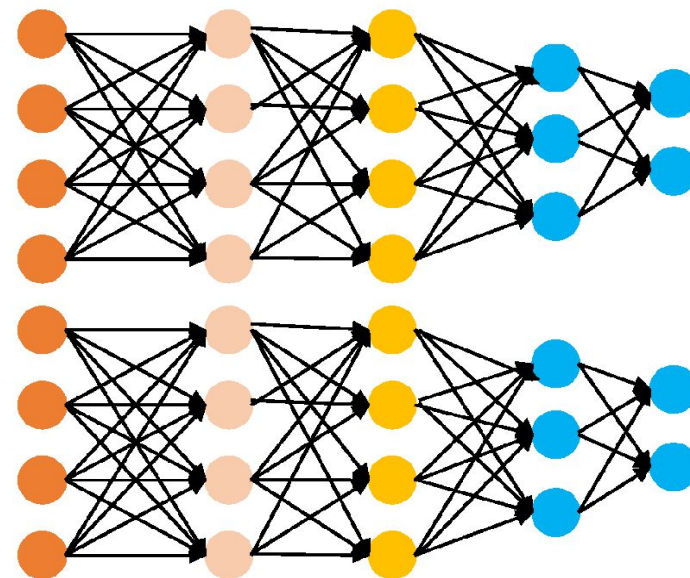
Controller



Consecutive images



Pre-processing stage



RL Decision Making

Actor



Critic

step-by-step meaning of every block in your diagram,
with the **exact functionality + core formulas**

One-line pipeline

$$s_t \rightarrow \text{CNN+features} \rightarrow z_t \rightarrow \begin{cases} \text{Actor: } a_t \\ \text{Critic: } V(z_t) \end{cases} \rightarrow \text{CARLA} \rightarrow (r_t, s_{t+1}) \rightarrow A_t \rightarrow \text{update A}$$

1) CARLA Environment (Top-left)

This is the driving simulator. At each time step t :

- It provides the state s_t (camera frames + other info)
- It receives the action a_t (steering/throttle/brake)
- It returns the reward r_t and next state s_{t+1}

$$\left(\mathbf{s}_t, \mathbf{a}_t \right) \rightarrow \left(\mathbf{r}_t, \mathbf{s}_{t+1} \right)$$

2) State s_t

So the state is not a single image—it is a stack / sequence:

$$s_t = \{I_{t-k+1}, I_{t-k+2}, \dots, I_t\}$$

This helps the agent understand motion (speed, direction, turning).

3) Pre-processing Stage (Bottom-left)

CNN Feature Extraction

Each image I_t is passed through a CNN:

$$h_t = (I_t)$$

Here:

- h_t = learned visual features (lane edges, vehicles, road curve, etc.)

4) RL Decision Making (Center networks)

The fused features z_t go into two networks:

4.1 Actor Network (Policy)

Actor produces the action distribution (or continuous action directly):

$$\pi(a_t | z_t; \theta) a_t = \mu(z_t; \theta)$$

➤ θ = actor parameters

4.2 Critic Network (Value)

Critic estimates “how good this state is”:

$$V(z_t; w)$$

➤ w = critic parameters

5) Action a_t and Controller (Top-right box)

The Actor output becomes the control command.

Typical continuous control:

$$a_t = (\delta_t, \tau_t, b_t)$$

Where:

- δ_t = steering
- τ_t = throttle
- b_t = brake

The controller applies this to the car (actuators).

6) Reward r_t (Top arrow)

After executing a_t , the environment returns a reward:

A common autonomous-driving reward is a weighted sum:

$$r_t = w_1 \cdot v_t - w_2 \cdot |e_t^{lane}| - w_3 \cdot \mathbb{1}() - w_4 \cdot \mathbb{1}() - w_5 \cdot |\Delta\delta_t|$$

Meaning:

- reward speed (progress)
- penalize lane deviation
- big penalty for collision/off-road
- penalize jerky steering (comfort)

7) Advantage / TD Error (Main learning logic)

The Critic helps compute the TD error (temporal-difference error):

$$\delta_t = r_t + \gamma V(\mathbf{z}_{t+1}; \mathbf{w}) - V(\mathbf{z}_t; \mathbf{w})$$

This is the “how surprising was the reward?” signal.

In Actor–Critic, we use Advantage (often $A_t \approx \delta_t$):

$$A_t = \delta_t$$

Or using n-step return (A3C commonly uses this):

$$R_t^{(n)} = \sum_{i=0}^{n-1} \gamma^i r_{t+i} + \gamma^n V(\mathbf{z}_{t+n})$$
$$A_t = R_t^{(n)} - V(\mathbf{z}_t)$$

8. Actor Loss (policy improvement)

We increase probability of good actions using advantage:

$$L_{actor}(\theta) = -\log \pi(a_t | z_t; \theta) A_t$$

Total

$$L = L_{actor} + cL_{critic} + L_{entropy}$$

